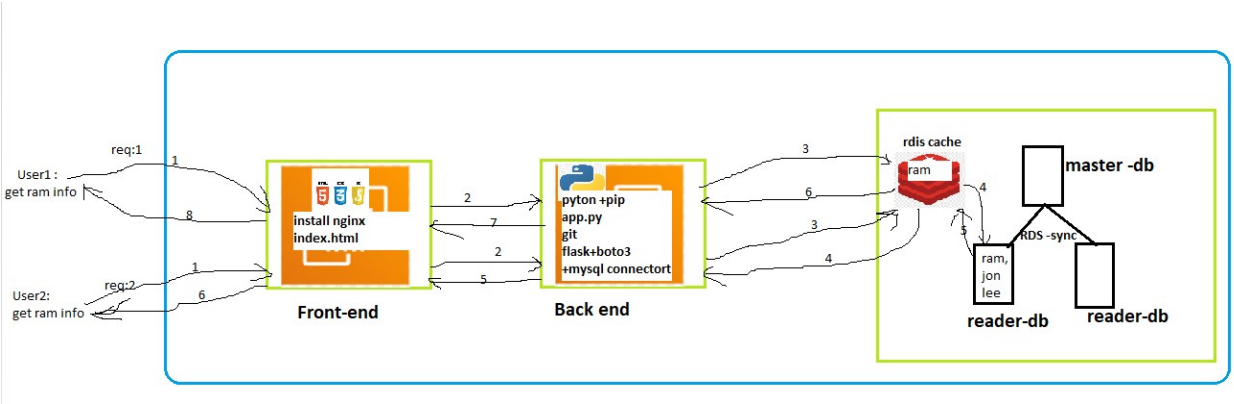
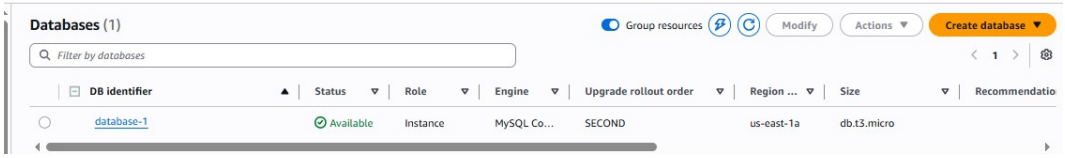
Deploy Backend using python , connect it to RDS DB with Redis Cache .

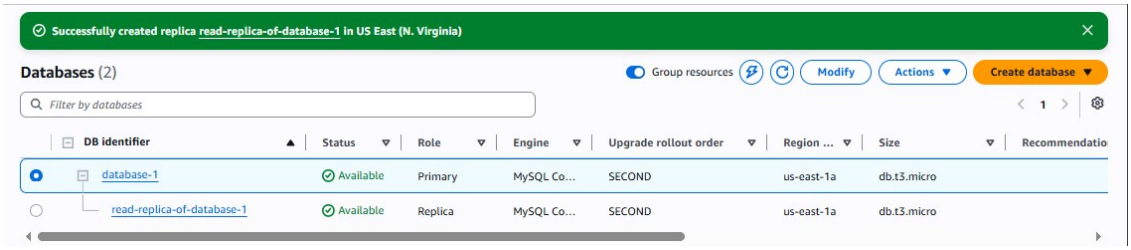


RDS Database configuration

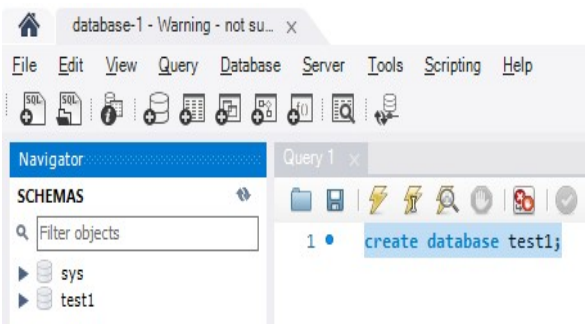
1. Create RDS database-1
   (public access + public subnet + self-managed password)

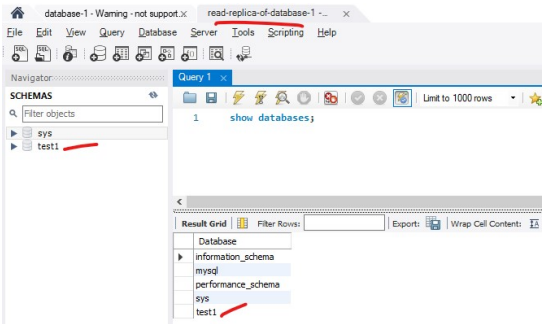


Create read replica . here we are creating it for testing purpose.

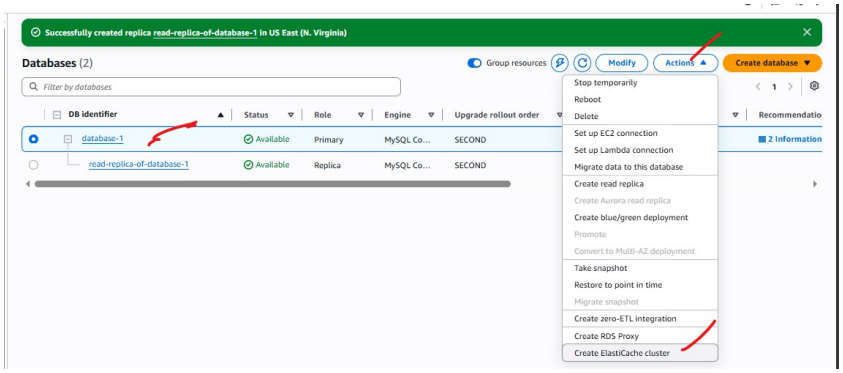


Try to connect with both RDS DB using **mysql workbench** to check connection

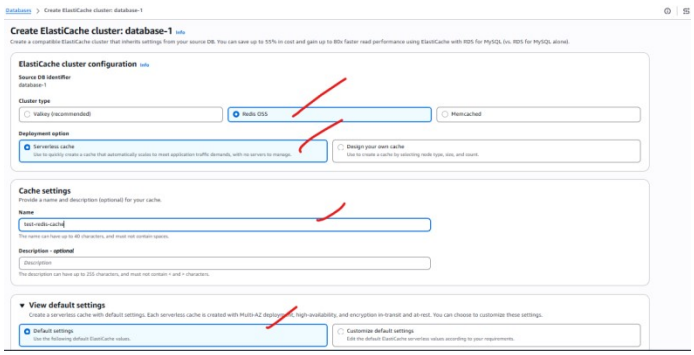




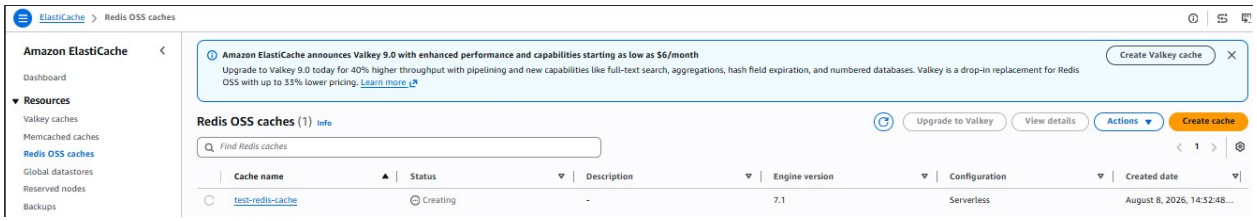
Create a chache for Primary/master DB (database-1).



Give a name , select options and create.

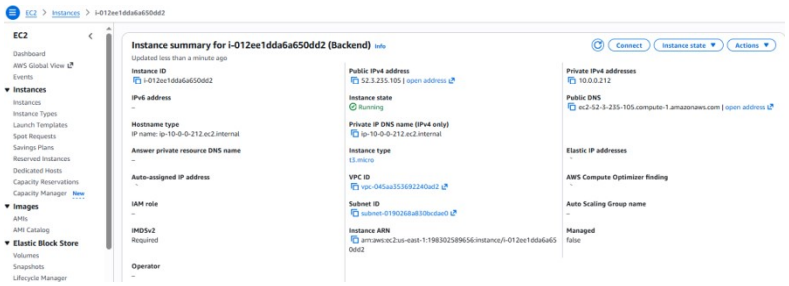


You can find the newly created cache under **Elastic Cache**



Backend configuration

Create public server and allow all traffic.



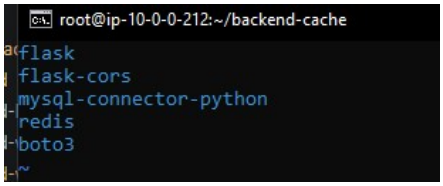
Switch to root >create and move to **backend-cache** directory>

-Install python + pip

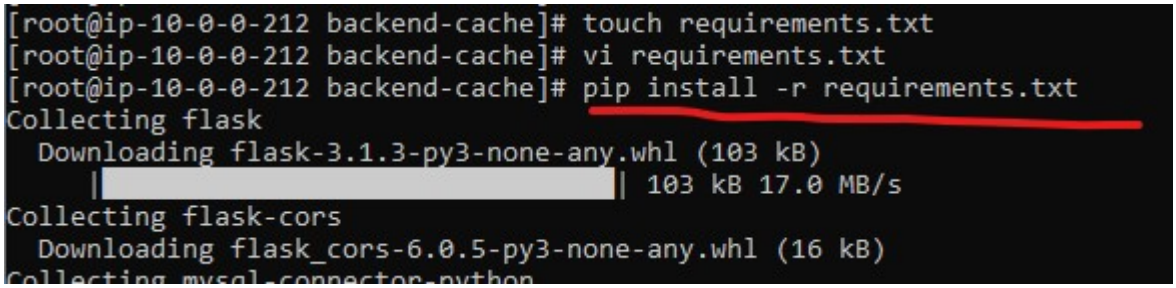
yum install python –y

yum install pip –y

-create a requirement.txt and include backend dependencies .

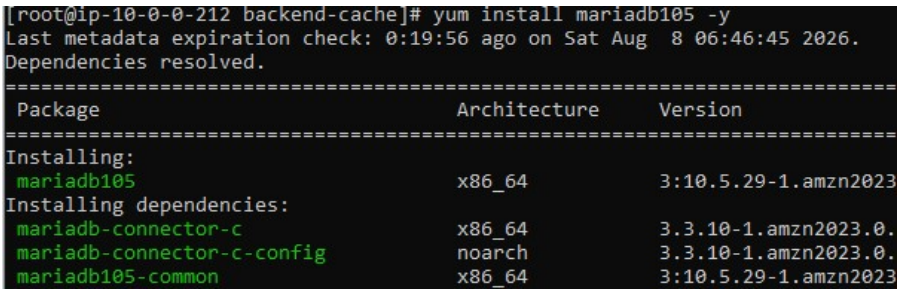


Then execute the command to install dependencies -> **pip install –r requirements.txt**



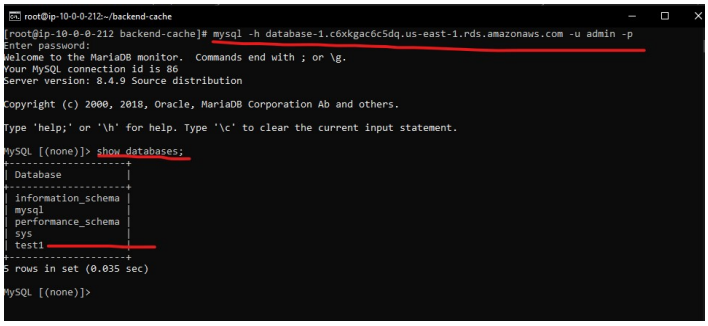
-we can use mysql agent to test , backend to RDS database-1 connection.

a. install agent -> **yum install mariadb105 -y**



b.use this to connect ->

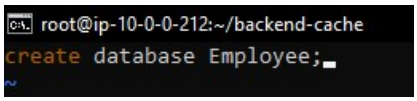
#mysql -h database-1.c6xkgac6c5dq.us-east-1.rds.amazonaws.com -u admin -p



****

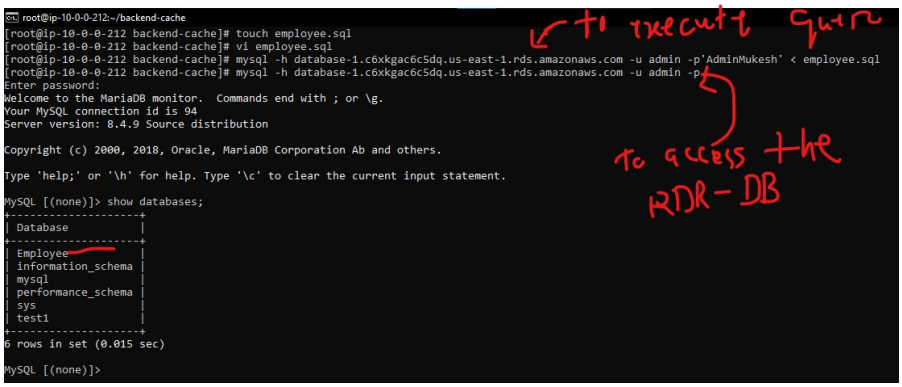
If you want to create or modify the **RDS database-1** staying in backend server by using mariadb105 , then you can create a file with **.sql** extension and include the query that you want to execute .

Got to backend-cache>Create employee.sql [include (create database Employee;)]



Now execute **->**

 # mysql -h database-1.c6xkgac6c5dq.us-east-1.rds.amazonaws.com -u admin -p'AdminMukesh' < employee.sql



Now create test.sql , include below code and execute .

```
CREATE DATABASE IF NOT EXISTS dev;
USE dev;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);

INSERT INTO users (name, email) VALUES
('John Doe', 'john@example.com'),
('Alice Smith', 'alice@example.com'),
('Bob Johnson', 'bob@example.com'),
('Eve Adams', 'eve@example.com');
```

```
[ec2-user@ip-10-0-0-212 ~]$ mysql -h database-1.c6xkgac6c5dq.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 107
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| Employee           |
| dev                |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| test1              |
+--------------------+
7 rows in set (0.001 sec)

MySQL [(none)]> use dev;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [dev]> select *from users;
+----+-------------+-------------------+
| id | name        | email             |
+----+-------------+-------------------+
|  1 | ram         | ram@example.com   |
|  2 | Alice Smith | alice@example.com |
|  3 | Bob Johnson | bob@example.com   |
|  4 | Eve Adams   | eve@example.com   |
+----+-------------+-------------------+
4 rows in set (0.002 sec)

MySQL [dev]>
```

-create **app.py** or **cache.py** and include backend code.

In that code Update the metrics

– 1. rds dns

-2 . reader –replica dns

-3 . redis chache dns

In redis cache section .

```python
# ========================
# REDIS CONFIG
# ========================
redis_client = redis.Redis(
    host='test-redis-cache-kkfh9h.serverless.use1.cache.amazonaws.com', #rdis dns
    port=6379,
    ssl=True,
    decode_responses=True,
    socket_timeout=5
)

CACHE_TTL = 90   #cache retention period
```

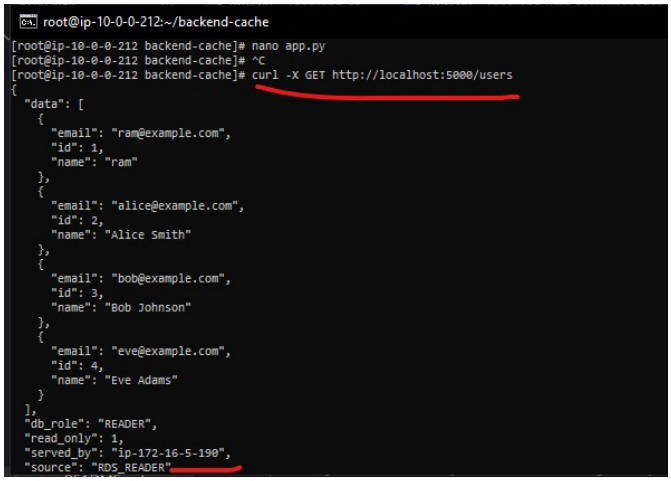
Get app.py code from cache.py - **AWS/26-8-3 Python and node deployment with RDS/python-backend-testing**

https://github.com/itmukesh-16/DevOps-With-Multicloud-/tree/main/AWS/26-8-3%20Python%20and%20node%20deployment%20with%20RDS/python-backend-testing

Then run -> **Python3 app.py**

Now run **curl -X Get and observe the source .**

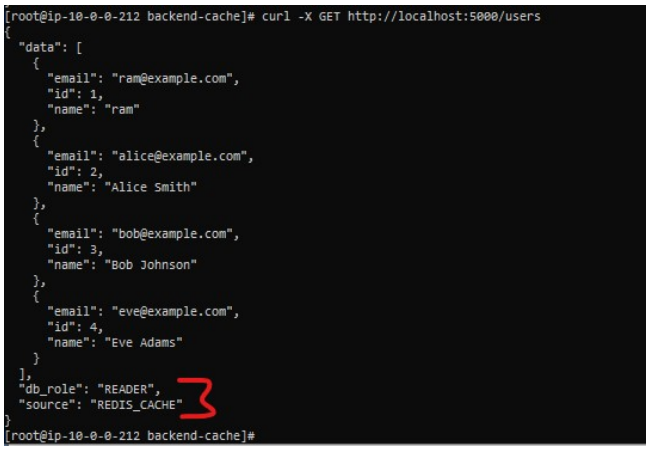
Initially the data fetched from RDS reader .



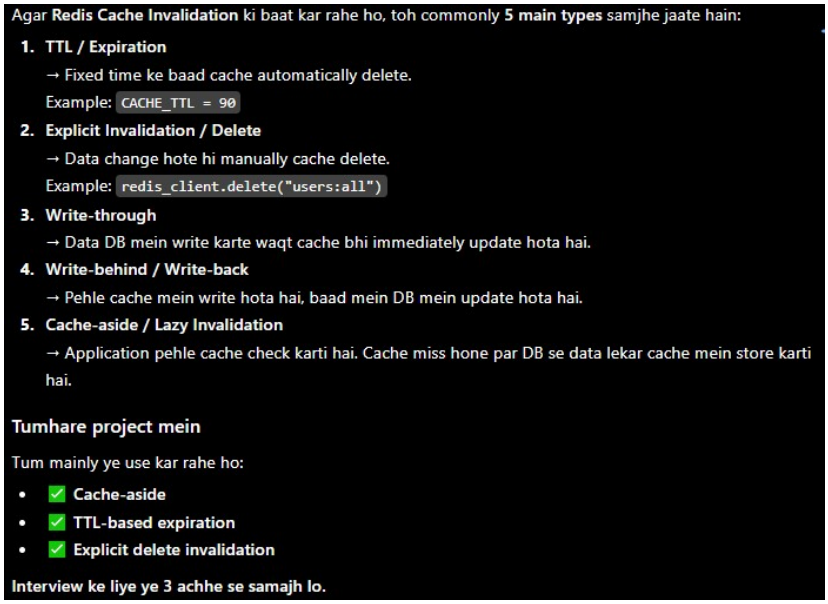
Then again get all user data and observe the source .

When the same data fetched multiple times , the frequently used data stored inside Redis Cache.

And in the next time when the same user or different user tried to access same data ,instead of going to read replica it fetched the data from the Redis cache .



Cache Invalidation –



Agar **Redis Cache Invalidation** ki baat kar rahe ho, toh commonly **5 main types** samjhe jaate hain:

1. **TTL / Expiration**
   → Fixed time ke baad cache automatically delete.
   Example: `CACHE_TTL = 90`
2. **Explicit Invalidation / Delete**
   → Data change hote hi manually cache delete.
   Example: `redis_client.delete("users:all")`
3. **Write-through**
   → Data DB mein write karte waqt cache bhi immediately update hota hai.
4. **Write-behind / Write-back**
   → Pehle cache mein write hota hai, baad mein DB mein update hota hai.
5. **Cache-aside / Lazy Invalidation**
   → Application pehle cache check karti hai. Cache miss hone par DB se data lekar cache mein store karti hai.

Tumhare project mein

Tum mainly ye use kar rahe ho:

- ✅ **Cache-aside**
- ✅ **TTL-based expiration**
- ✅ **Explicit delete invalidation**

Interview ke liye ye 3 achhe se samajh lo.

Frontend configuration

Now we can create **Frontend server** and install **nginx** .

in inside **/usr/share/nginx/html/index.html** folder give your front end index.html file

we can configure without proxy or with reverse-proxy.

We can also configure ROUTE53 for the Domain .